Name: Kriti
College: Lovely Professional University
Week6 Assignment
Email: kritijulia@gmail.com
StudentId:CT_CSI_DV_5113

Q1. Deploy Replica Set and Replication Controller, and deployment. Also learn the advantages and disadvantages of each.

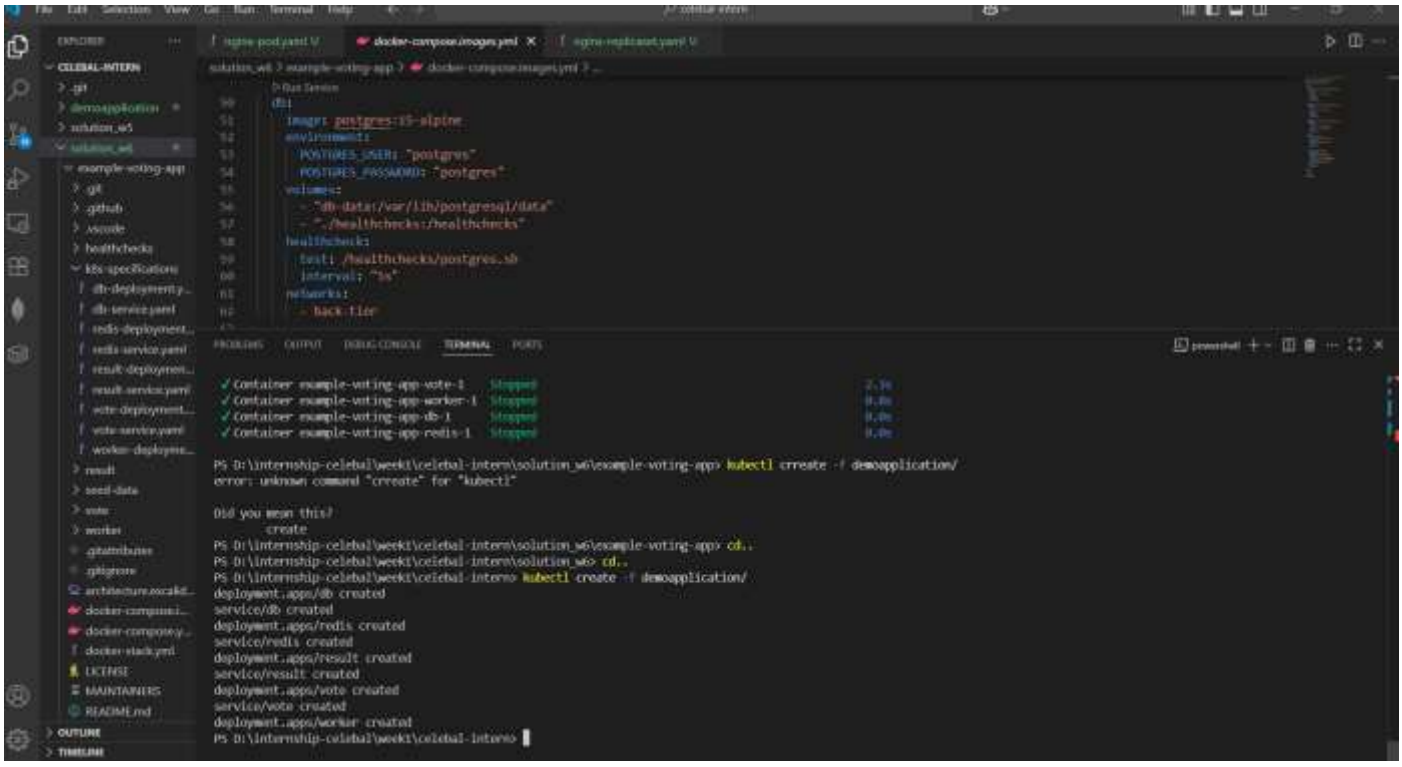
Answer:

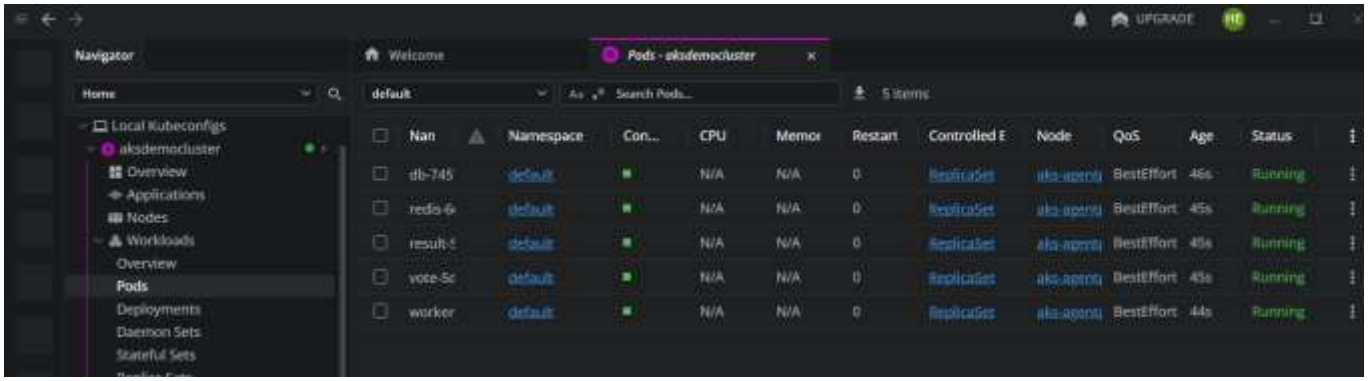
Feature	ReplicationController	ReplicaSet	Deployment
Introduced in	Kubernetes v1 (legacy)	Replacement for RC	Manages ReplicaSets
Rolling Updates	NO	NO	Yes
Rollbacks	NO	NO	Yes
Label Selector	Equality only	Set-based	Set-based
Usage in Modern K8s	Deprecated	Rarely used directly	Standard
Versioning Support	NO	NO	Yes

1. ReplicaSet (RS) : Ensures that a specified number of **pod replicas** are running at any given time.
    a. Advantages:
        i. Maintains the **desired number of pod replicas**.
        ii. Supports **set-based label selectors** (more powerful than ReplicationController).
    b. Disadvantages:
        i. **Cannot handle rolling updates or rollbacks.**
        ii. Not typically used directly — usually managed through a Deployment.
    c. Commands:
        i. Deploy: kubectl apply -f nginx-replicaset.yaml
        ii. Check status: kubectl get rs
        iii. Delete: kubectl delete -f nginx-replicaset.yaml
2. ReplicationController (RC): The **older** version of ReplicaSet, used to ensure a specific number of pod replicas are running.
    a. Advantages:
        i. Ensures **availability** by maintaining pod replicas.
        ii. Simpler and well-supported in older Kubernetes versions

b. Disadvantages:
    i. Limited **label selector** support (only equality-based)
    ii. .**Deprecated** in favor of ReplicaSet and Deployment.
    iii. **No support for rolling updates or rollbacks.**
c. Commands:
    i. Deploy: kubectl apply -f nginx-rc.yaml
    ii. Check status: kubectl get rc
    iii. Delete: kubectl delete -f nginx-rc.yaml

3. Deployment: Manages ReplicaSets and provides **rolling updates**, **rollback**, and **version control** for applications.
    a. Advantages:
        i. **Rolling updates** without downtime.
        ii. **Rollbacks** to previous versions easily.
        iii. Manages **ReplicaSets** automatically.
        iv. Preferred method for most use cases.
    b. Disadvantages:
        i. Slightly more complex than directly using ReplicaSet.
        ii. Not suitable if fine-grained control of ReplicaSet is required.
    c. Commands:
        i. Deploy: kubectl apply -f nginx-deployment.yaml
        ii. Check status: kubectl get deploy
        iii. Delete: kubectl delete -f nginx-deployment.yaml

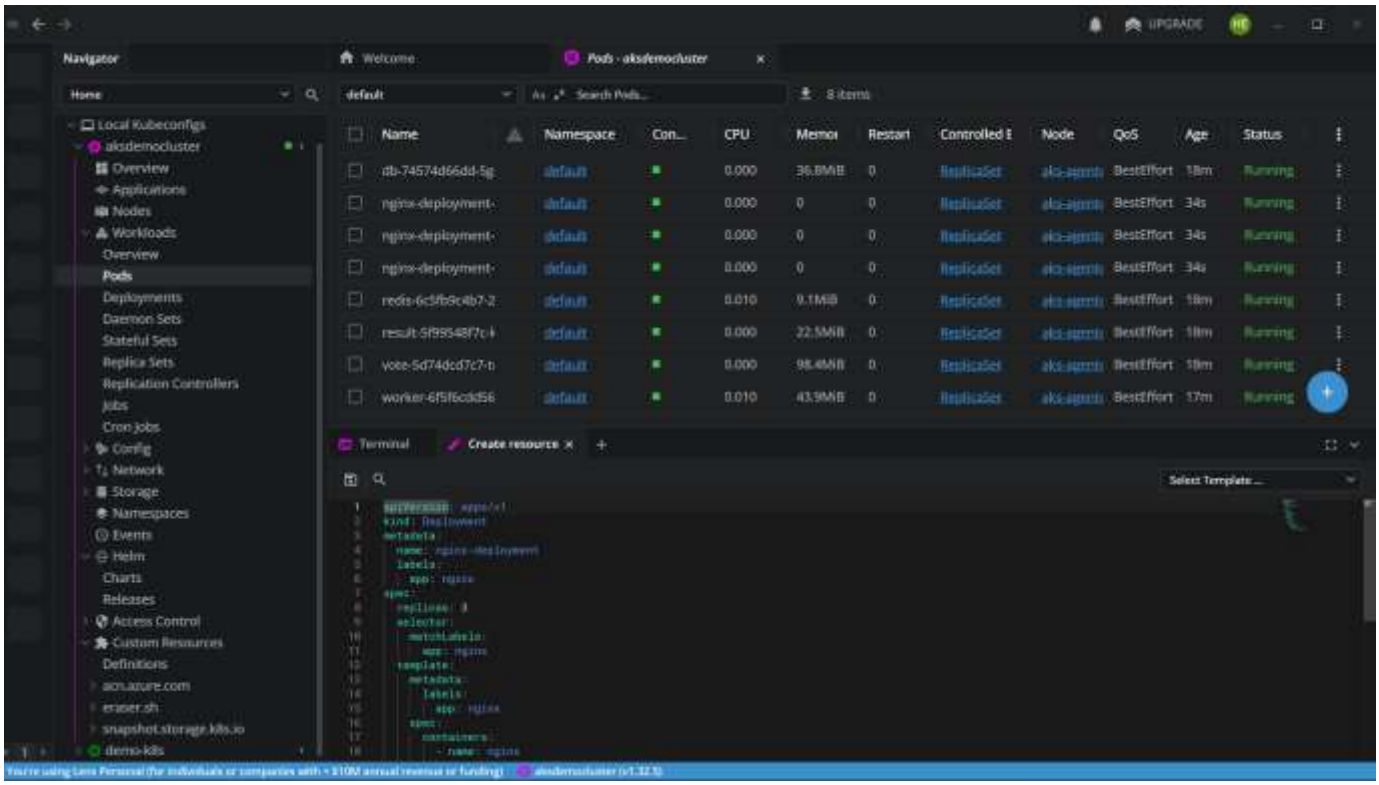


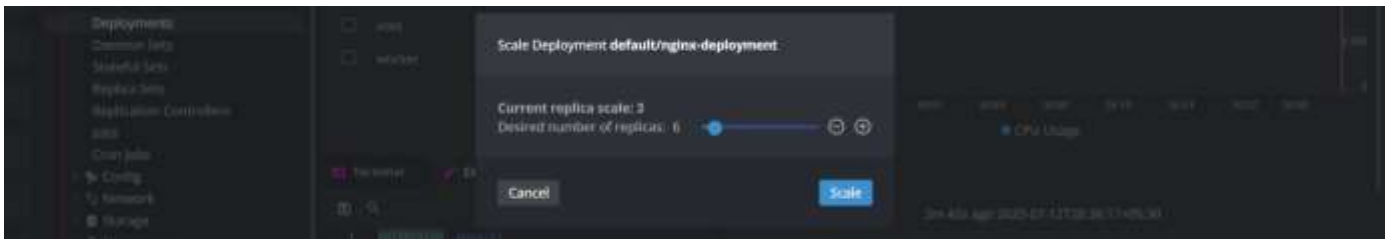


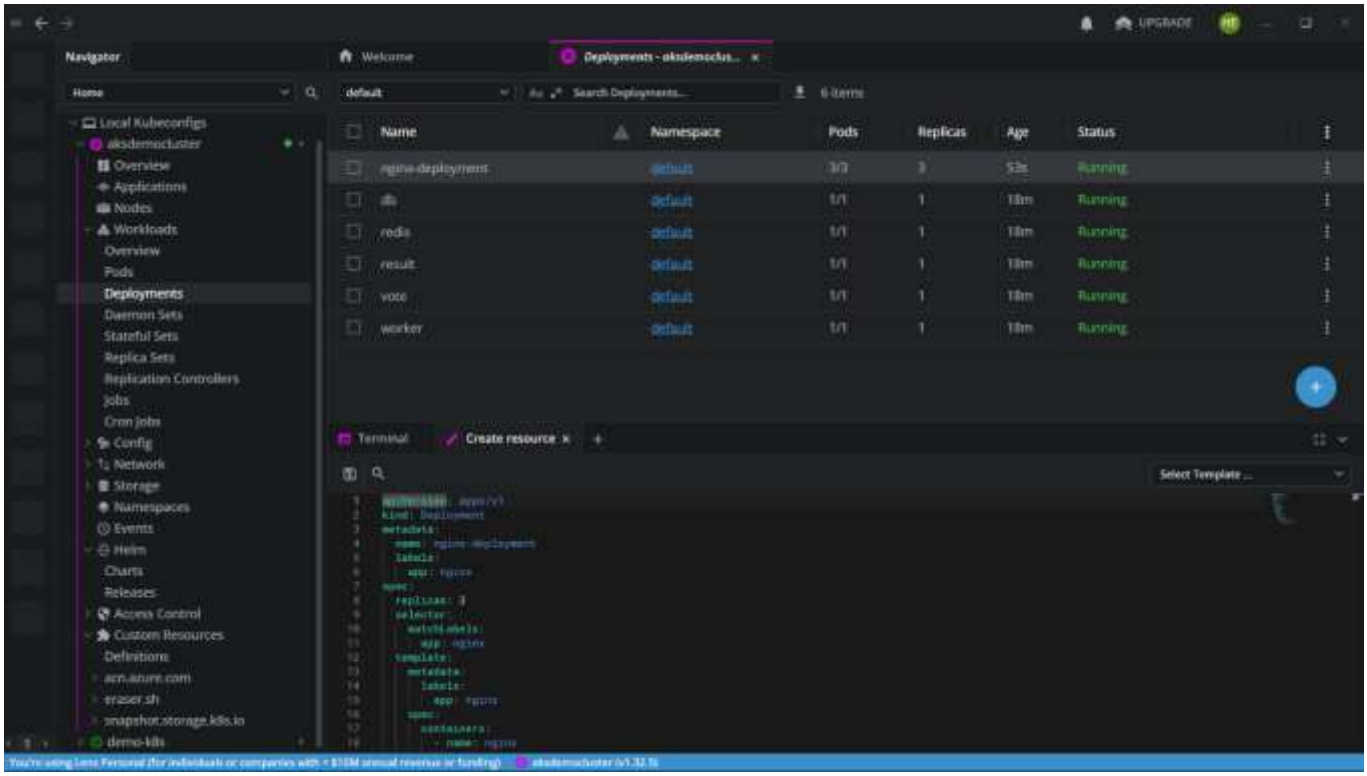

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: frontend
  labels:
    app: guestbook
    tier: frontend
spec:
  # modify replicas according to your case
  replicas: 3
  selector:
    matchLabels:
      tier: frontend
  template:
    metadata:
      labels:
        tier: frontend
    spec:
      containers:
        - name: php-redis
          image: gcr.io/google_samples/gb-frontend:v3
```

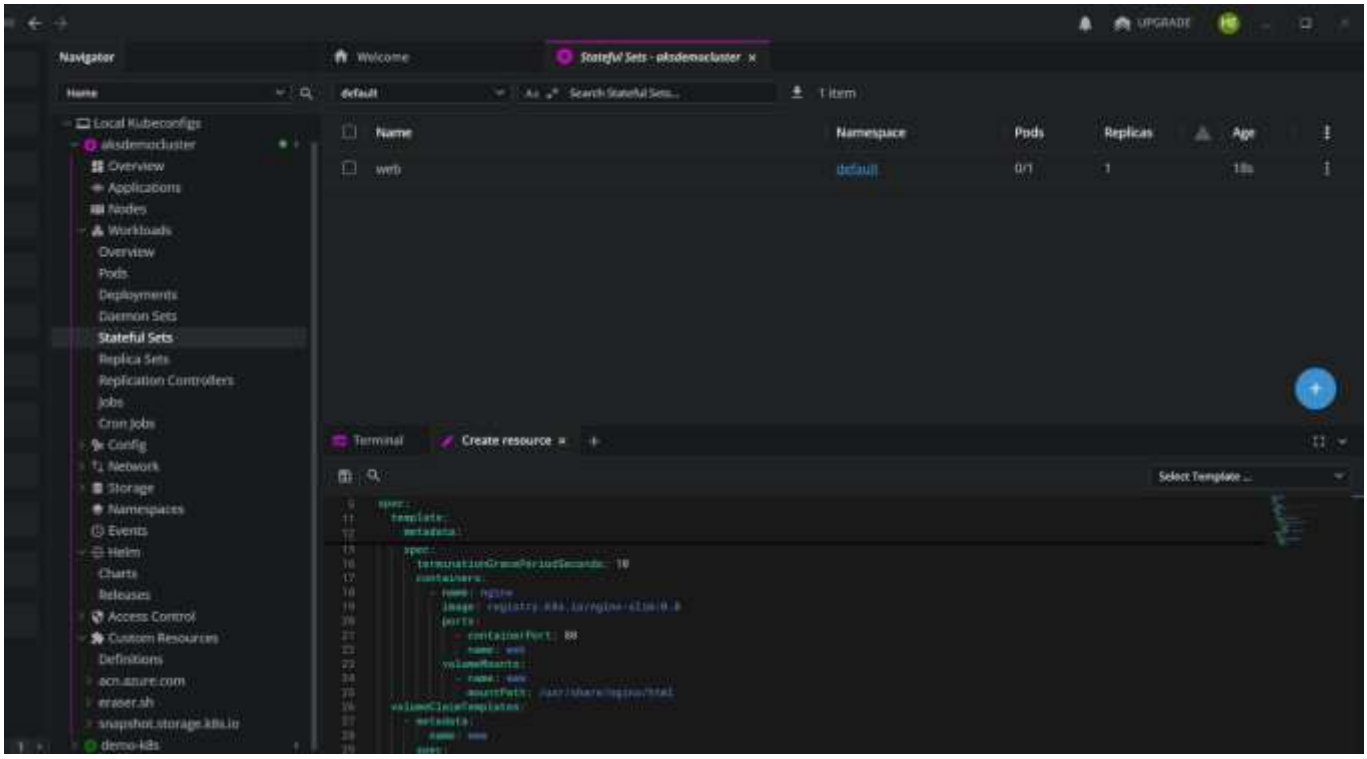
```yaml
apiVersion: v1
kind: ReplicationController
metadata:
  name: nginx
spec:
  replicas: 3
  selector:
    app: nginx
  template:
    metadata:
      name: nginx
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx
          ports:
            - containerPort: 80
```

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.14.2
          ports:
            - containerPort: 80
```






Navigator
Welcome
Deployments - aksdemocluster
Home
default
Search Deployments...
6 items
Name
Namespace
Pods
Replicas
Age
Status
nginx-deployment
default
3/3
3
53s
Running
default
1/1
1
18m
Running
redis
default
1/1
1
18m
Running
result
default
1/1
1
18m
Running
vote
default
1/1
1
18m
Running
worker
default
1/1
1
18m
Running
Local Kubeconfigs
aksdemocluster
Overview
Applications
Nodes
Workloads
Overview
Pods
Deployments
Daemon Sets
Stateful Sets
Replica Sets
Replication Controllers
Jobs
Cron Jobs
Config
Network
Storage
Namespaces
Events
Helm
Charts
Releases
Access Control
Custom Resources
Definitions
acn.azure.com
eraser.sh
snapshot.storage.k8s.io
demo-k8s
Terminal
Create resource
Select Template ...
apiVersion: apps/v1
kind: Deployment
metadata:
name: nginx-deployment
labels:
app: nginx
spec:
replicas: 3
selector:
matchLabels:
app: nginx
template:
metadata:
labels:
app: nginx
spec:
containers:
- name: nginx

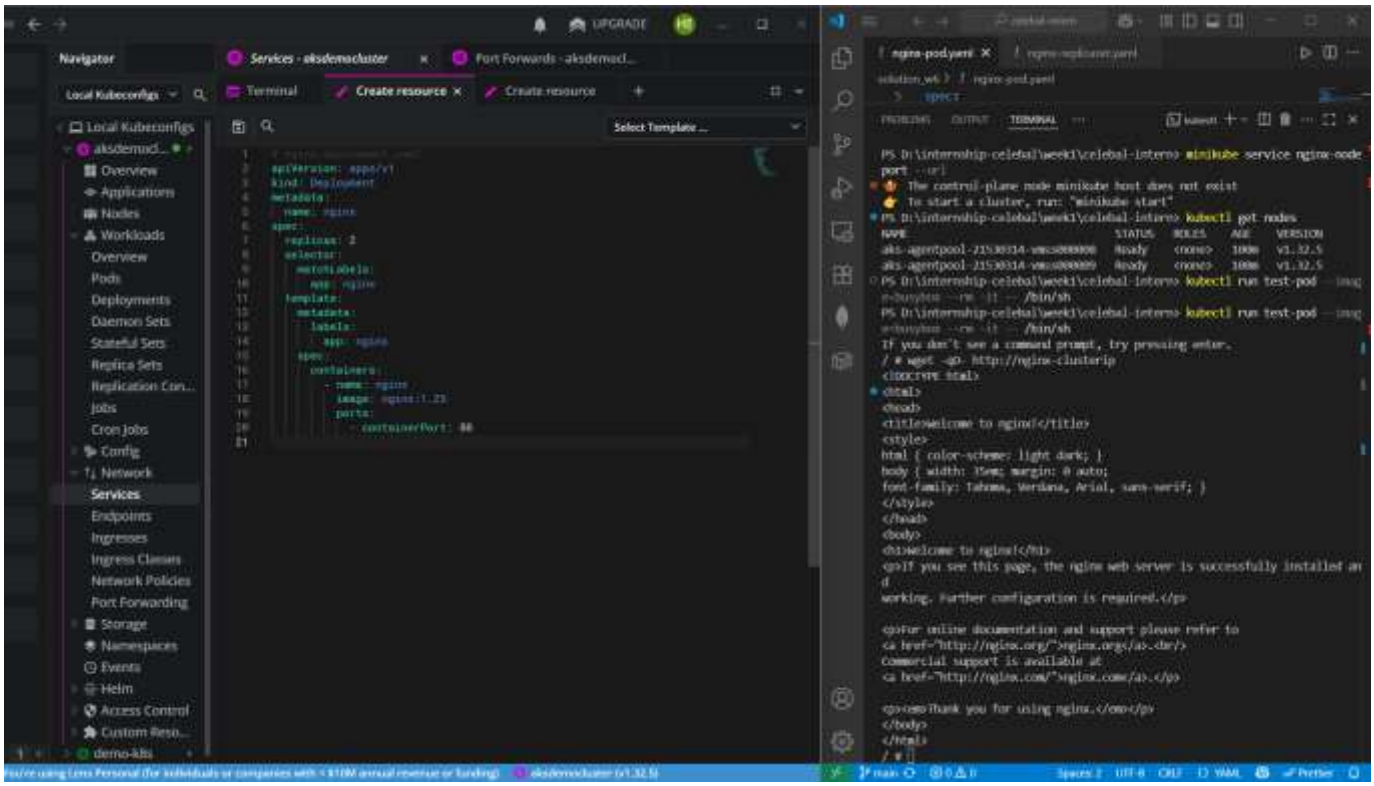

Navigator
Welcome
Stateful Sets - aksdemocluster
Home
default
Search Stateful Sets...
1 item
Name
Namespace
Pods
Replicas
Age
web
default
0/1
1
18s
Local Kubeconfigs
aksdemocluster
Overview
Applications
Nodes
Workloads
Overview
Pods
Deployments
Daemon Sets
Stateful Sets
Replica Sets
Replication Controllers
Jobs
Cron Jobs
Config
Network
Storage
Namespaces
Events
Helm
Charts
Releases
Access Control
Custom Resources
Definitions
acn.azure.com
eraser.sh
snapshot.storage.k8s.io
demo-k8s
Terminal
Create resource
Select Template ...
spec:
template:
metadata:
spec:
terminationGracePeriodSeconds: 10
containers:
- name: nginx
image: registry.k8s.io/nginx-slim:0.8
ports:
- containerPort: 80
name: web
volumeMounts:
- name: www
mountPath: /usr/share/nginx/html
volumeClaimTemplates:
- metadata:
name: www

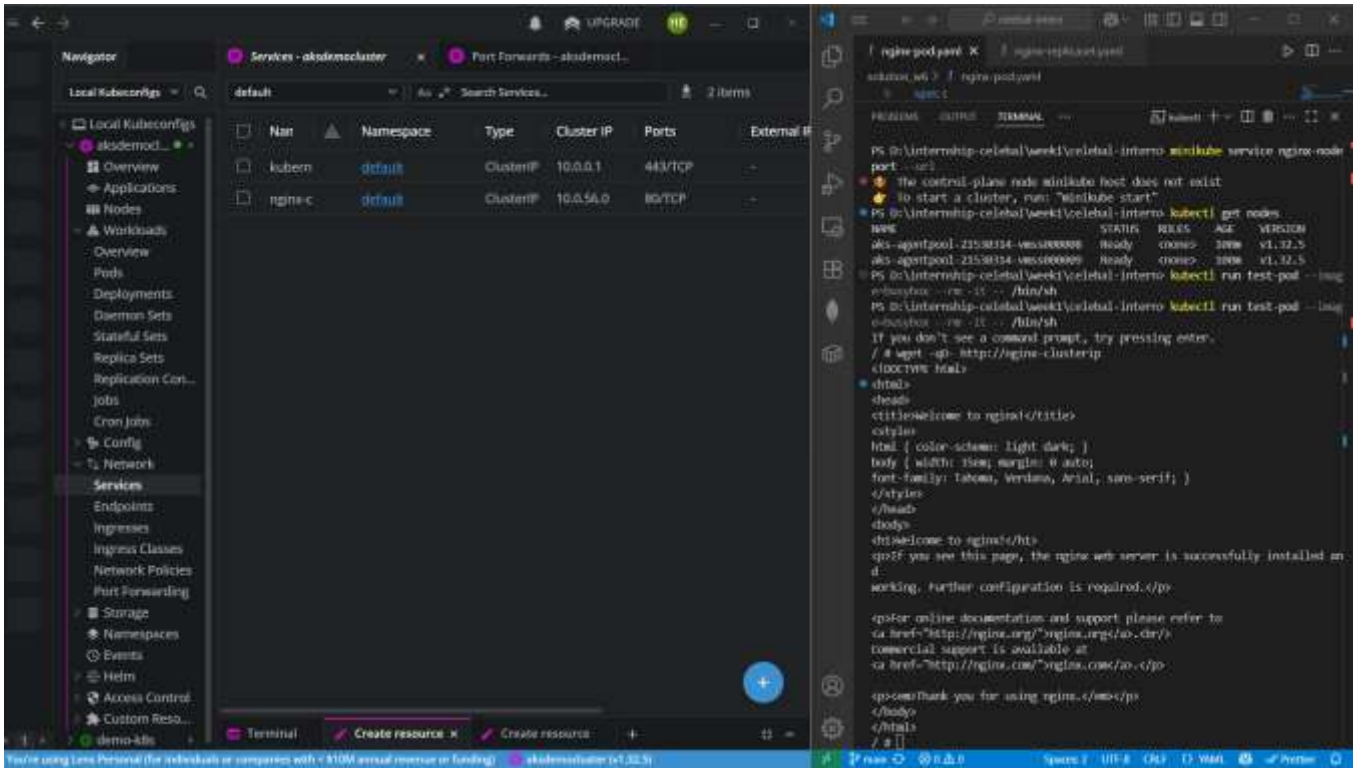
Q2. Kubernetes service types (ClusterIP, NodePort, LoadBalancer)

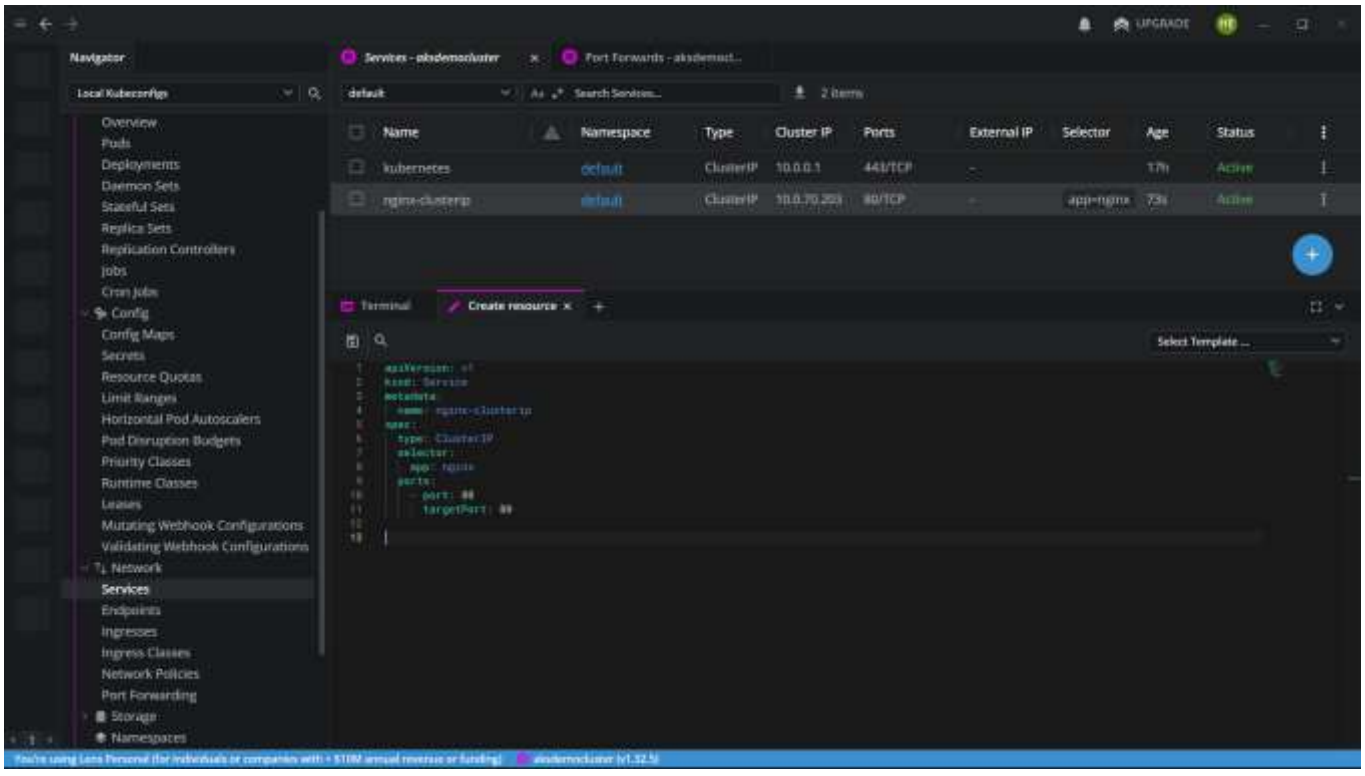
Answer:

In Kubernetes, Services provide a stable network identity and DNS name for a set of pods. There are three main types of services for exposing applications:

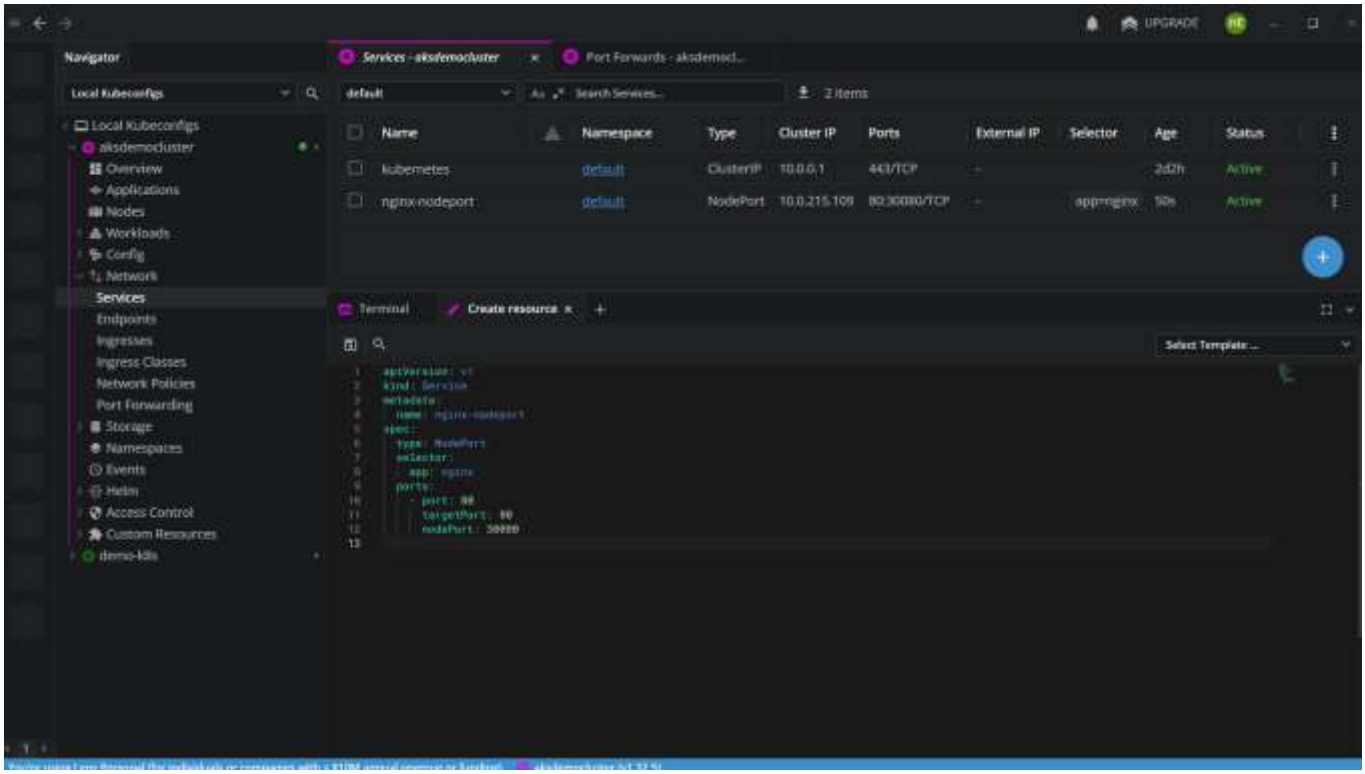
1. **ClusterIP (default):**
    a. Exposes the service internally (within the cluster only).
    b. Pods can communicate with it, but not accessible from outside the cluster.
    c. Used for Internal microservices, backend-to-backend communication.





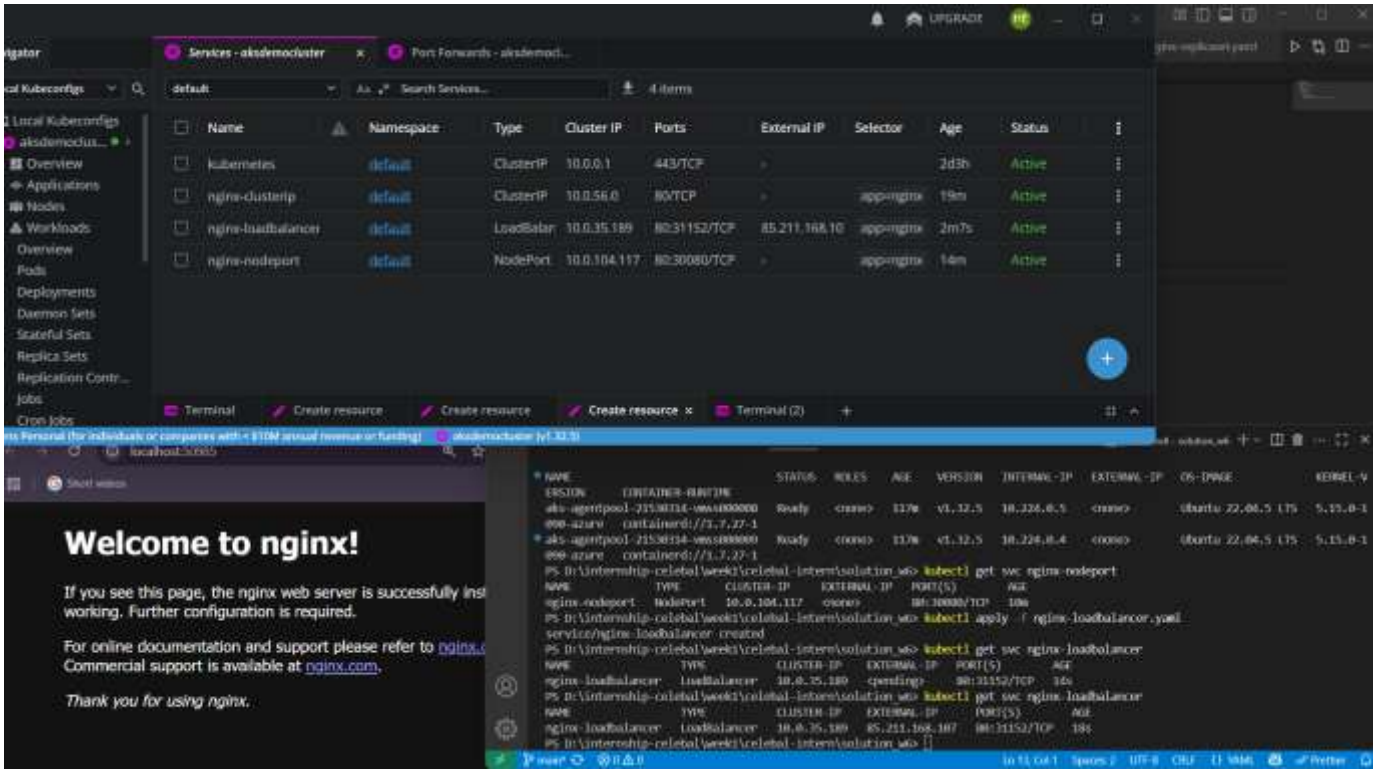


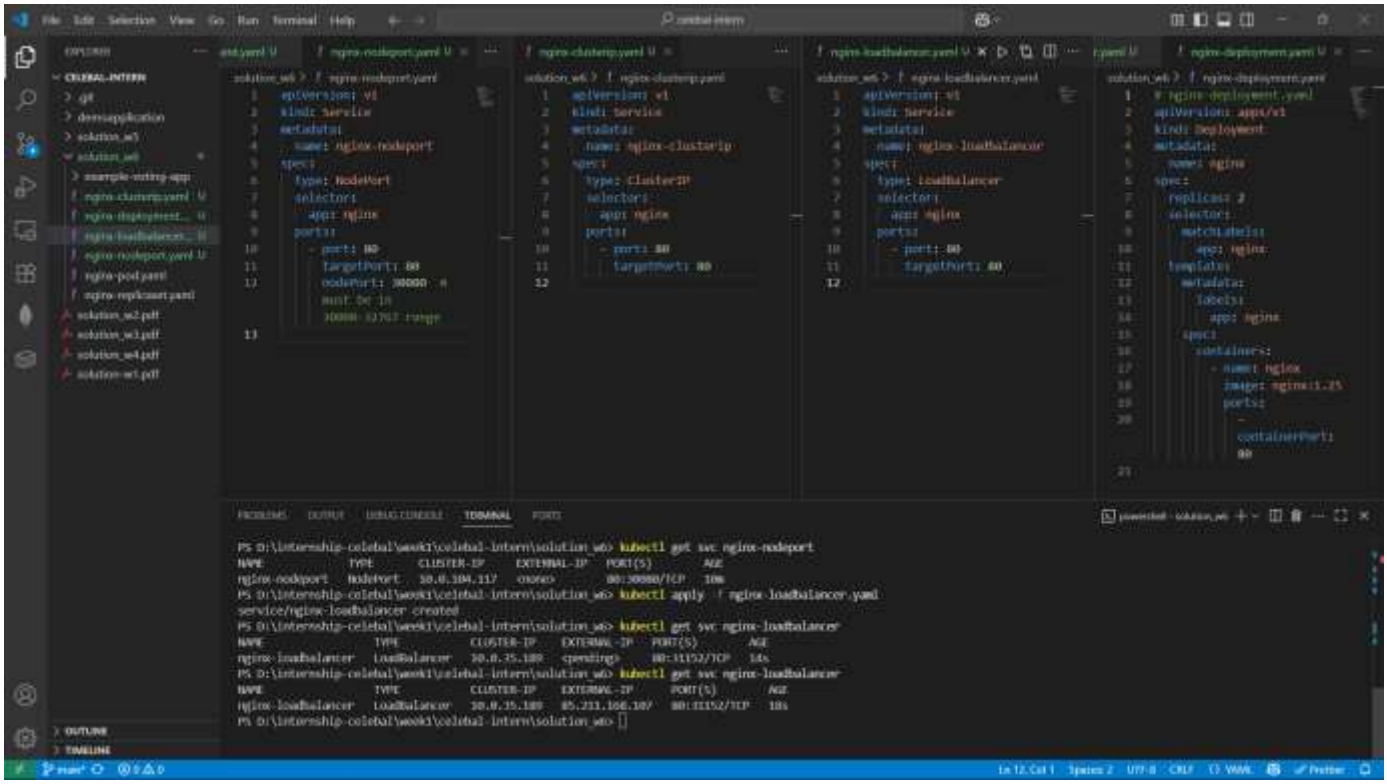
2. **NodePort:**
    a. Exposes the service on each Node's IP at a static port (30000–32767).
    b. **Makes the service accessible from outside the cluster using.**
    c. **http://<NodeIP>:<NodePort>**
    d. **For development/testing environments where you want to quickly access an app from outside.**



3. LoadBalancer:

a. **Provisions an external load balancer** (supported in cloud platforms like Azure, AWS, GCP).
b. Exposes service with a **public IP**.
c. Automatically routes traffic to backend pods.
d. Production-grade apps that need public access.





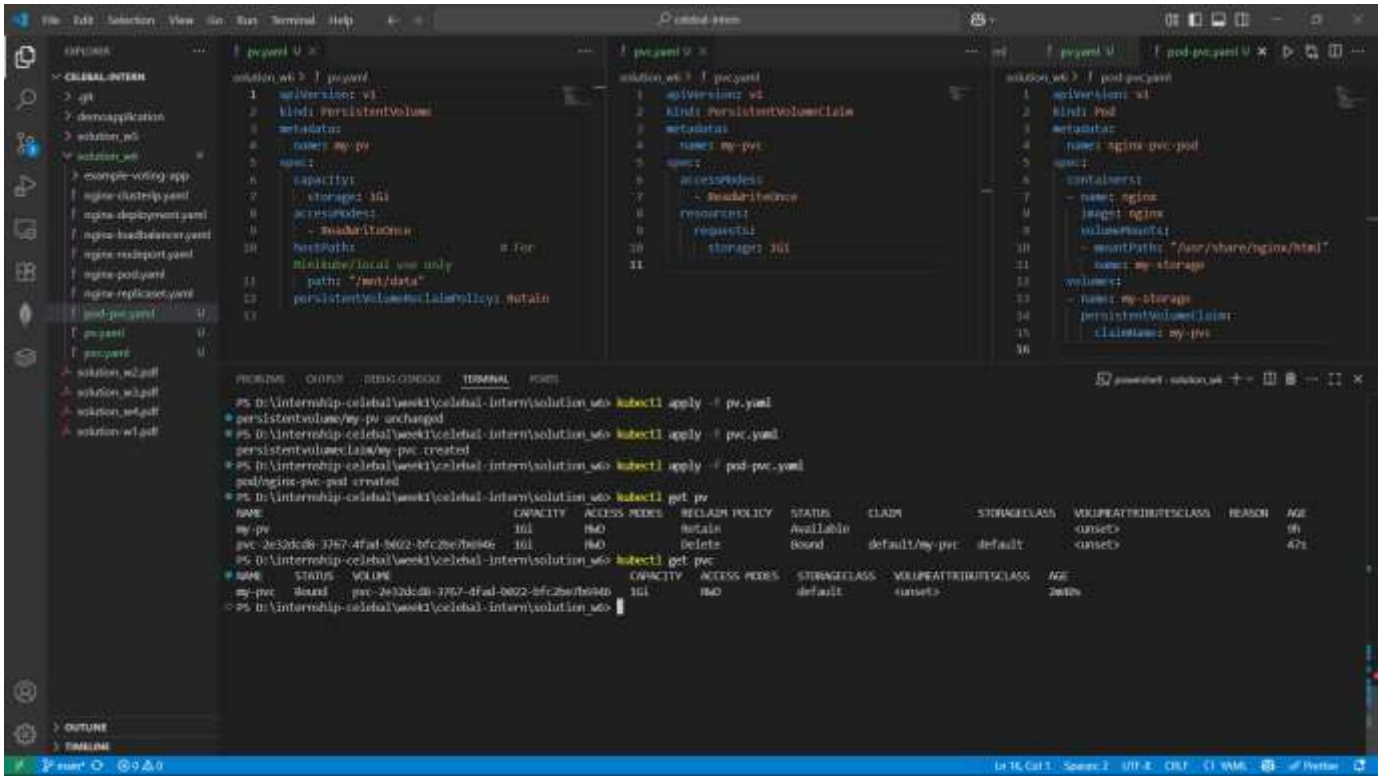
Q3. PersistentVolume (PV) and PersistentVolumeClaim (PVC)

What is a PersistentVolume (PV)?

A PersistentVolume is a piece of storage provisioned by an admin or dynamically provisioned using a StorageClass. It’s like a USB drive available to the cluster.

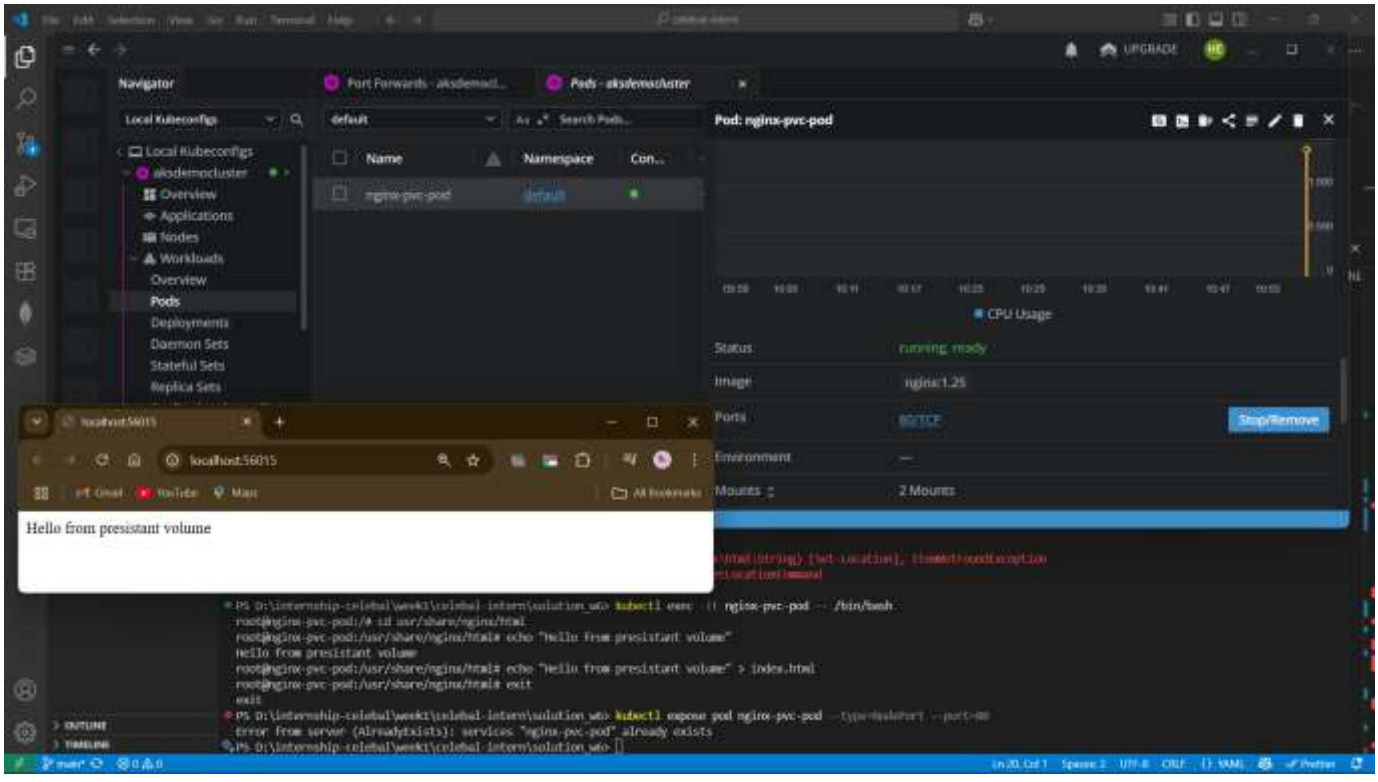
What is a PersistentVolumeClaim (PVC)?

A PersistentVolumeClaim is a request for storage by a user (pod). Think of it like:
"I want 1Gi of storage that supports ReadWriteOnce".


pv.yaml
apiVersion: v1
kind: PersistentVolume
metadata:
name: my-pv
spec:
capacity:
storage: 1Gi
accessModes:
- ReadWriteOnce
hostPath: # For Minikube/local use only
path: "/mnt/data"
persistentVolumeReclaimPolicy: Retain
pvc.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
name: my-pvc
spec:
accessModes:
- ReadWriteOnce
resources:
requests:
storage: 1Gi
pod-pvc.yaml
apiVersion: v1
kind: Pod
metadata:
name: nginx-pvc-pod
spec:
containers:
- name: nginx
image: nginx
volumeMounts:
- mountPath: "/usr/share/nginx/html"
name: my-storage
volumes:
- name: my-storage
persistentVolumeClaim:
claimName: my-pvc
PS D:\internship-celebal\week1\celebal-intern\solution_w6> kubectl apply -f pv.yaml
persistentvolume/my-pv unchanged
PS D:\internship-celebal\week1\celebal-intern\solution_w6> kubectl apply -f pvc.yaml
persistentvolumeclaim/my-pvc created
PS D:\internship-celebal\week1\celebal-intern\solution_w6> kubectl apply -f pod-pvc.yaml
pod/nginx-pvc-pod created
PS D:\internship-celebal\week1\celebal-intern\solution_w6> kubectl get pv
NAME CAPACITY ACCESS MODES RECLAIM POLICY STATUS CLAIM STORAGECLASS VOLUMEATTRIBUTESCLASS REASON AGE
my-pv 1Gi RWO Retain Available <unset> 9h
pvc-2e32dcd8-3767-4fad-b022-bfc2be7b6940 1Gi RWO Delete Bound default/my-pvc default <unset> 47s
PS D:\internship-celebal\week1\celebal-intern\solution_w6> kubectl get pvc
NAME STATUS VOLUME CAPACITY ACCESS MODES STORAGECLASS VOLUMEATTRIBUTESCLASS AGE
my-pvc Bound pvc-2e32dcd8-3767-4fad-b022-bfc2be7b6940 1Gi RWO default <unset> 3m49s
PS D:\internship-celebal\week1\celebal-intern\solution_w6>


Navigator
Port Forwards - aksdemoc...
Pods - aksdemocluster
Local Kubeconfigs
default
1 item
Name
Namespace
Con...
CPU
Memor
Restart
Controlled E
Node
QoS
Age
Status
nginx-pvc-pod
default
0.000
3.1MiB
0
BestEffort
119s
Running
Local Kubeconfigs
aksdemocluster
Overview
Applications
Nodes


Navigator
Port Forwards - aksdemoc...
Persistent Volume Claims - a...
Local Kubeconfigs
default
1 item
Name
Namespace
Storage class
Size
Pods
Age
Status
my-pvc
default
default
1Gi
nginx-pvc
3m47s
Bound
Local Kubeconfigs
aksdemocluster
Overview
Applications


Navigator
Port Forwards - aksdemoc...
Pods - aksdemocluster
Pod: nginx-pvc-pod
Local Kubeconfigs
default
Name
Namespace
Con...
nginx-pvc-pod
default
Overview
Applications
Nodes
Workloads
Overview
Pods
Deployments
Daemon Sets
Stateful Sets
Replica Sets
CPU Usage
Status
running, ready
Image
nginx:1.25
Ports
80/TCP
Stop/Remove
Environment
Mounts
2 Mounts
localhost:56015
Hello from presistant volume
PS D:\internship-celebal\week1\celebal-intern\solution_w6> kubectl exec -it nginx-pvc-pod -- /bin/bash
root@nginx-pvc-pod:/# cd usr/share/nginx/html
root@nginx-pvc-pod:/usr/share/nginx/html# echo "Hello from presistant volume"
Hello from presistant volume
root@nginx-pvc-pod:/usr/share/nginx/html# echo "Hello from presistant volume" > index.html
root@nginx-pvc-pod:/usr/share/nginx/html# exit
exit
PS D:\internship-celebal\week1\celebal-intern\solution_w6> kubectl expose pod nginx-pvc-pod --type=NodePort --port=80
Error from server (AlreadyExists): services "nginx-pvc-pod" already exists
PS D:\internship-celebal\week1\celebal-intern\solution_w6>

Q4. Managing Kubernetes with Azure Kubernetes Service (AKS), Creating and managing AKS clusters, Scaling and upgrading AKS clusters

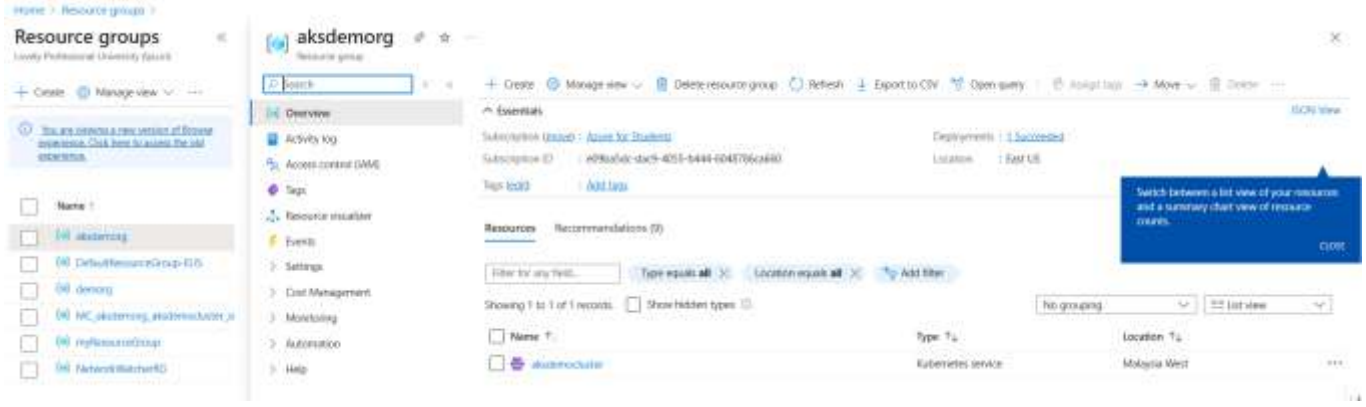
AKS (Azure Kubernetes Service) is Microsoft's fully managed Kubernetes service.
Think of it like this:

"You want to use Kubernetes but don't want to handle all the hard setup, updates, and server maintenance — Azure does it for you."

Step1: **Creating an AKS Cluster or we can use azure command line also**

Using Azure Portal (GUI)

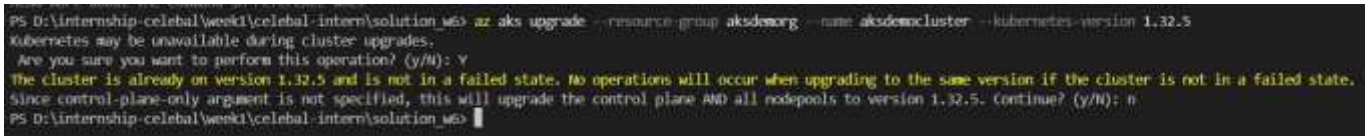
1. Go to https://portal.azure.com
2. Search for **"Kubernetes services"** → Click **Create**
3. Fill in details:
    - **Resource Group**: Like a folder for resources.
    - **Cluster Name**: e.g., my-aks-cluster
    - **Region**: e.g., Central India
    - **Node Size & Count**: Like choosing how powerful and how many VMs you want.
4. Hit **Review + Create**, and Azure will set it all up for you in a few minutes.



Step2: Managing the Cluster

Step3: Scaling the Cluster means we can set it as autoscaling or scaling. In autoscaling we can scale it by giving it minimum and maximum nodes so that when ever new node is required by the aks it can create without any interference. But in scaling we manually add new node to manage the load.

Step4: Upgrading to latest version
note: Azure does a **rolling upgrade**, so your workloads keep running without downtime.



Q5. Configure liveliness and readiness probes for pods in AKS cluster.

Before we start, let's understand the **why**.

- **Liveness Probe** checks **if the app is still running**. If it's not, Kubernetes restarts the container.
- **Readiness Probe** checks **if the app is ready to serve requests**. If it's not ready, Kubernetes **removes the pod from service load balancers** until it is.

This helps keep apps highly available, stable, and self-healing.

Step 1: Connect to the AKS Cluster

First, I connect my terminal to the AKS cluster using the following command:

```
az aks get-credentials --resource-group myResourceGroup --name myAKSCluster
```

This command downloads the credentials and sets my kubectl context to AKS.

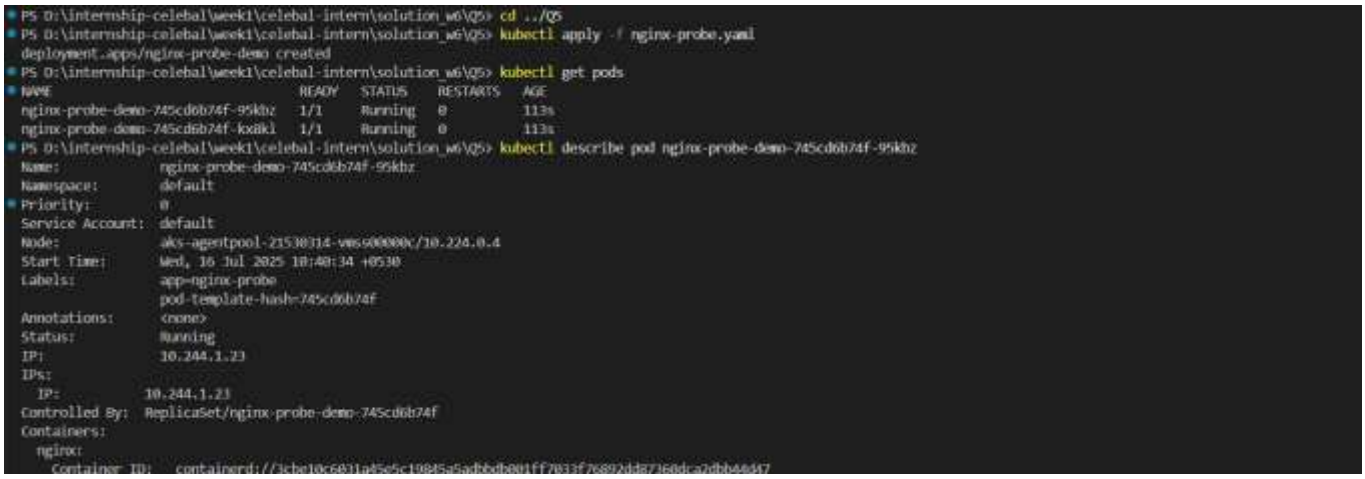
Step 2: Create a Deployment with Probes

Now, I create a new deployment named nginx-probe-demo. But this time, I'm going to add **both liveness and readiness probes** in the YAML definition.

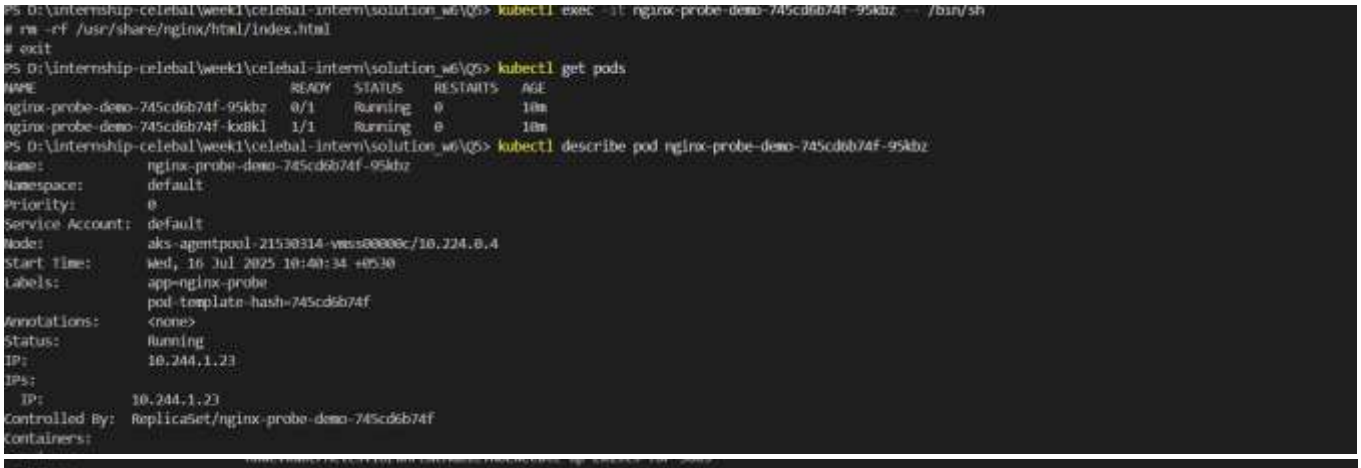
This mean the **livenessProbe** starts checking 10 seconds after the container starts and repeats every 5 seconds. If it fails, the container is restarted and the **readinessProbe** checks after 5 seconds to see if the container is ready to serve traffic.

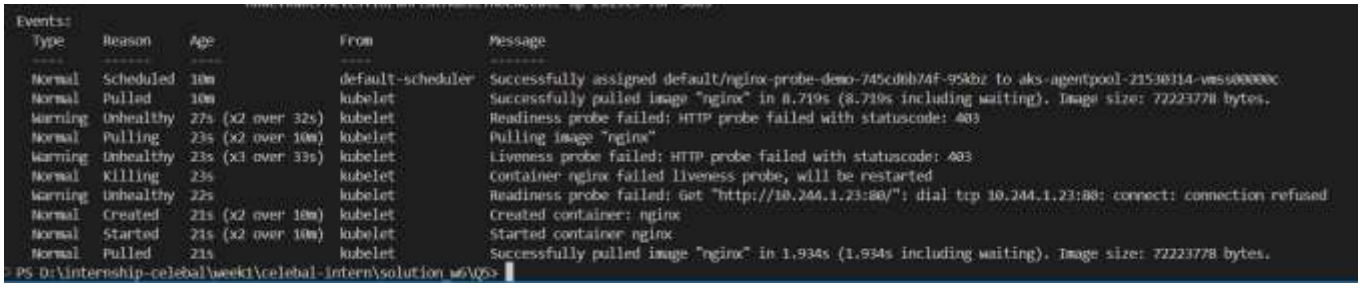
Step 3: Apply the Deployment

Step 4: Monitor the Probes in Action : If the **readiness probe failed**, the pod won't receive traffic and If the **liveness probe failed**, the pod will be restarted. In the output, I check the events section. It will show me:



Step5 : Simulate Failure - To test the probe in action, I could run the below command and Then observe the logs and pod restarts





- With liveness probes, Kubernetes ensures that crashed containers get restarted.

- With readiness probes, Kubernetes ensures that traffic goes only to healthy pods.
- These probes are critical in production environments to maintain high uptime and graceful scaling.

Q6. Configure Taints and Tolerants.

What Are Taints and Tolerations?

Let me explain this simply:

- A **taint** is something we apply to a **node** to say:

"Don't schedule any pods here unless they can tolerate this condition."

- A **toleration** is something we apply to a **pod** to say:

"It's okay, I can run on a node with this taint."

This helps us **control which pods can run on which nodes**, useful for:

- Running sensitive workloads separately
- Isolating GPU/ML nodes
- Separating environments (e.g., dev vs prod)

Step 1: Get Your Node Names - First, I list the nodes in my AKS cluster: “kubectl get nodes”. I pick one of the node names,

Step 2: Apply a Taint to the Node

```
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q5> kubectl get nodes
NAME                                STATUS   ROLES    AGE   VERSION
aks-agentpool-21530314-vmss00000c   Ready    <none>   38m   v1.32.5
aks-agentpool-21530314-vmss00000d   Ready    <none>   38m   v1.32.5
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q5> kubectl taint nodes aks-agentpool-21530314-vmss00000c key=exclusive:Noschedule
error: invalid taint effect: Noschedule, unsupported taint effect
See 'kubectl taint -h' for help and examples
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q5> kubectl taint nodes aks-agentpool-21530314-vmss00000c key=exclusive:NoSchedule
node/aks-agentpool-21530314-vmss00000c tainted
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q5>
```

This tells Kubernetes **not to schedule pods on this node** unless they **tolerate** key=exclusive

Step 3: Try Running a Pod Without Toleration

```
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q6> kubectl apply -f pod-no-toleration.yaml
pod/nginx-no-toleration created
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q6> kubectl get pods
NAME                                READY   STATUS    RESTARTS      AGE
nginx-no-toleration                 1/1     Running   0             13s
nginx-probe-demo-745cd6b74f-95kbz   1/1     Running   1 (8m40s ago)   18m
nginx-probe-demo-745cd6b74f-ko8kl   1/1     Running   0             18m
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q6> kubectl describe pod nginx-no-toleration
Name:             nginx-no-toleration
Namespace:        default
Priority:         0
Service Account:  default
Node:             aks-agentpool-21530314-vmss00000d/10.224.0.5
Start Time:       Wed, 16 Jul 2025 10:59:07 +0530
Labels:           <none>
Annotations:      <none>
Status:           Running
IP:               10.244.0.104
IPs:
  IP:  10.244.0.104
Containers:
  nginx:
    Container ID:   containerd://ab0804c504750d28b9fd397d5673fddeb44243d4a405130f474ec65e554d4e1e
    Image:          nginx
    Image ID:       docker.io/library/nginx@sha256:f5c017fb33c6db484545793ffb67db51cdd7daebee472104612f73a85063f889
```

```
Node-Selectors:              <none>
Tolerations:                 node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                             node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type    Reason     Age   From               Message
  ----    ------     ----  ----               -------
  Normal  Scheduled  13s   default-scheduler  Successfully assigned default/nginx-no-toleration to aks-agentpool-21530314-vmss00000d
  Normal  Pulling    13s   kubelet            Pulling image "nginx"
  Normal  Pulled     11s   kubelet            Successfully pulled image "nginx" in 1.914s (1.914s including waiting). Image size: 72223778 bytes.
  Normal  Created    11s   kubelet            Created container: nginx
  Normal  Started    11s   kubelet            Started container nginx
PS D:\internship-celebal\week1\celebal-intern\solution w6\Q6>
```

Output of kubectl describe pod showing the **"node(s) had taints"** error.

Step 4: Add Toleration to Allow Scheduling

Output of kubectl get pods -o wide showing the pod running

```
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q6> kubectl apply -f pod-with-toleration.yaml
>>
pod/nginx-with-toleration created
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q6> kubectl get pods -o wide
NAME                                READY   STATUS    RESTARTS      AGE     IP             NODE                                NOMINATED NODE   READINESS GATES
nginx-no-toleration                 1/1     Running   0             4m28s   10.244.0.104   aks-agentpool-21530314-vmss00000d   <none>           <none>
nginx-probe-demo-745cd6b74f-95kbz   1/1     Running   1 (12m ago)   23m     10.244.1.23    aks-agentpool-21530314-vmss00000c   <none>           <none>
nginx-probe-demo-745cd6b74f-ko8kl   1/1     Running   0             23m     10.244.0.205   aks-agentpool-21530314-vmss00000d   <none>           <none>
nginx-with-toleration               1/1     Running   0             20s     10.244.0.232   aks-agentpool-21530314-vmss00000d   <none>           <none>
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q6> kubectl describe pod nginx-with-toleration
Name:             nginx-with-toleration
Namespace:        default
Priority:         0
Service Account:  default
Node:             aks-agentpool-21530314-vmss00000d/10.224.0.5
Start Time:       Wed, 16 Jul 2025 11:03:15 +0530
Labels:           <none>
Annotations:      <none>
Status:           Running
IP:               10.244.0.232
IPs:
  IP:  10.244.0.232
Containers:
  nginx:
    Container ID:   containerd://ea680ec402f077c815089a4421d5e5e0e972da03317dbc601b3ebc858d0055fc
    Image:          nginx
    Image ID:       docker.io/library/nginx@sha256:f5c017fb33c6db484545793ffb67db51cdd7daebee472104612f73a85063f889
    Port:           <none>
    Host Port:      <none>
    State:          Running
```

```
Node-Selectors:              <none>
Tolerations:                 exclusive-true:NoSchedule
                             node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                             node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type    Reason     Age   From               Message
  ----    ------     ----  ----               -------
  Normal  Scheduled  35s   default-scheduler  Successfully assigned default/nginx-with-toleration to aks-agentpool-21530314-vmss00000d
  Normal  Pulling    35s   kubelet            Pulling image "nginx"
  Normal  Pulled     33s   kubelet            Successfully pulled image "nginx" in 1.912s (1.912s including waiting). Image size: 72223778 bytes.
  Normal  Created    33s   kubelet            Created container: nginx
  Normal  Started    33s   kubelet            Started container nginx
```

Output of kubectl describe pod nginx-with-toleration

Step 5: Remove Taint (Clean Up)

```
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q6> kubectl taint nodes aks-agentpool-21530314-vmss00000c key=exclusive:NoSchedule-
node/aks-agentpool-21530314-vmss00000c untainted
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q6>
```

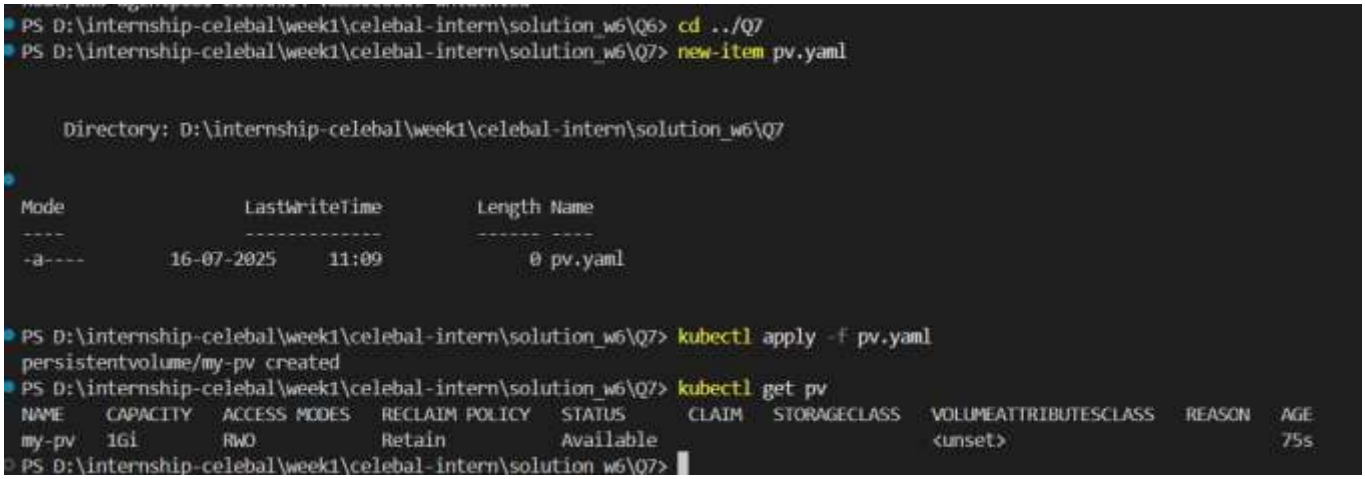
- I **tainted** a node so that pods won't go there unless they tolerate it.
- I showed how a pod without toleration fails to schedule.
- I added a **toleration** to a new pod and showed that it runs fine on the tainted node.
- This is useful when we want **strict control over pod placement**, like for **security, performance, or isolation**.

Q7. Create and attach persistent volume claims to pods.

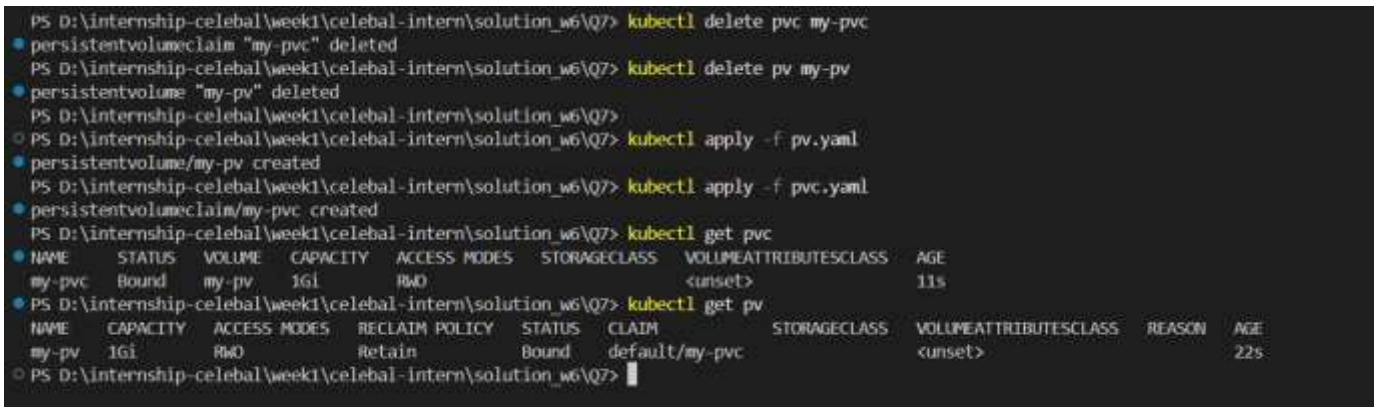
magine you're running an app that stores data — like logs, user uploads, or configs. If the pod crashes or restarts, that data will be lost unless you attach persistent storage.

A PersistentVolumeClaim (PVC) lets your pod request and use persistent storage.

Step 1: Create a Persistent Volume (PV) - This step defines **where** the storage physically exists.



Step 2: Create a Persistent Volume Claim (PVC)



Step 3: Attach PVC to a Pod

Let's check if the volume is mounted and working.

```
QoS Class:                   BestEffort
Node-Selectors:              <none>
Tolerations:                 node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                             node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type    Reason     Age   From               Message
  ----    ------     ----  ----               -------
  Normal  Scheduled  20s   default-scheduler  Successfully assigned default/pvc-nginx to aks-agentpool-21530314-vmss00000d
  Normal  Pulling    20s   kubelet            Pulling image "nginx"
  Normal  Pulled     18s   kubelet            Successfully pulled image "nginx" in 1.837s (1.837s including waiting). Image size: 72223778 bytes.
  Normal  Created    18s   kubelet            Created container: nginx
  Normal  Started    18s   kubelet            Started container nginx
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q7> kubectl exec -it pvc-nginx -- /bin/bash
root@pvc-nginx:/# cat index.html
cat: index.html: No such file or directory
root@pvc-nginx:/# cd /usr/share/nginx/html
root@pvc-nginx:/usr/share/nginx/html# echo "Hello from PVC" > index.html
root@pvc-nginx:/usr/share/nginx/html# cat index.html
Hello from PVC
```

Terminal showing file creation and content from mounted volume

Q8. Configure health probes for pods.

When you're running applications in Kubernetes, you need a way to **check if your application is running properly**.

Kubernetes uses two types of health checks (probes):

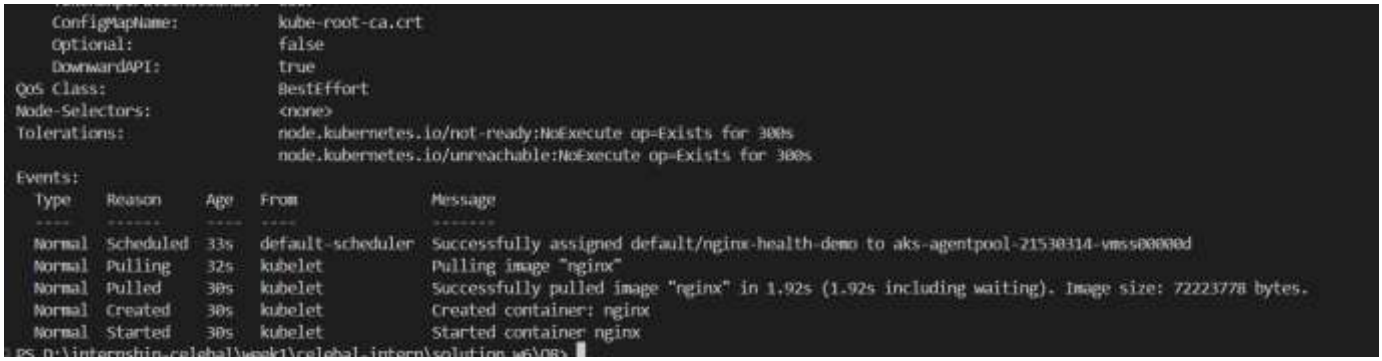
- **Liveness Probe**:
  Checks **if the app is still alive**. If it fails, Kubernetes **restarts** the container.
- **Readiness Probe**:
  Checks **if the app is ready to serve traffic**. If it fails, the pod is **removed from the load balancer** temporarily.

This keeps your app **self-healing** and **resilient**.

Step 1: Create a Pod with Liveness and Readiness Probe

I'm creating an NGINX pod that exposes a simple HTTP service, and I'm adding both types of health probes to it.

```
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q7> cd ../Q8
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q8> kubectl apply -f nginx-probes.yaml
pod/nginx-health-demo created
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q8> kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
nginx-health-demo   1/1     Running   0          26s
pvc-nginx           1/1     Running   0          6m2s
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q8> kubectl describe pod nginx-health-demo
Name:             nginx-health-demo
Namespace:        default
Priority:         0
Service Account:  default
Node:             aks-agentpool-21530314-vmss00000d/10.224.0.5
Start Time:       Wed, 16 Jul 2025 12:38:24 +0530
Labels:           <none>
Annotations:      <none>
Status:           Running
IP:               10.244.0.12
IPs:
```



Liveness Probe starts after 10s and runs every 5s. If it fails, the container restarts.

Readiness Probe starts after 5s and runs every 5s. If it fails, the pod is marked "Not Ready" and is temporarily removed from load balancing.

Q9. Configure autoscaling in your cluster (Horizontal scaling)

Horizontal Pod Autoscaler (HPA) automatically adjusts the number of pod replicas based on **CPU usage** or other custom metrics.

Think of it like this:

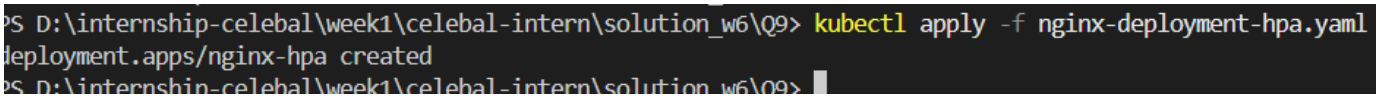
- When traffic is low, run fewer pods to save resources.
- When traffic spikes, automatically increase the number of pods to handle it.

HPA helps your app scale **dynamically**, without manual intervention.

Step 1: Enable Metrics Server (Required)

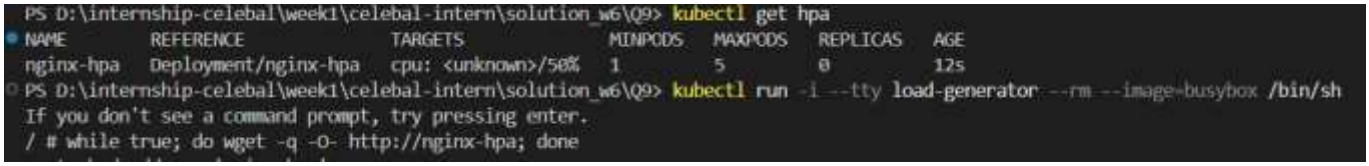
Step 2: Create a Sample Deployment with CPU Limits

We'll use a simple NGINX deployment but add CPU **requests and limits** — required for autoscaling.



Step 3: Create the Horizontal Pod Autoscaler

Step 4: View HPA Status



Step 5: Simulate High CPU Load

Step 6: Stop Load and Watch Scale Down

```
PS D:\internship-celebal\week1\celebal-intern> kubectl get hpa
NAME        REFERENCE              TARGETS       MINPODS   MAXPODS   REPLICAS   AGE
nginx-hpa   Deployment/nginx-hpa   cpu: 0%/50%   1         5         1          4m52s
PS D:\internship-celebal\week1\celebal-intern> kubectl get pods
NAME                          READY   STATUS    RESTARTS   AGE
load-generator                1/1     Running   0          41s
nginx-health-demo             1/1     Running   0          9m58s
nginx-hpa-5b6dcf667b-tzx4g    1/1     Running   0          6m4s
pvc-nginx                     1/1     Running   0          15m
PS D:\internship-celebal\week1\celebal-intern> kubectl get hpa
NAME        REFERENCE              TARGETS       MINPODS   MAXPODS   REPLICAS   AGE
nginx-hpa   Deployment/nginx-hpa   cpu: 0%/50%   1         5         1          4m53s
PS D:\internship-celebal\week1\celebal-intern> kubectl get pods
NAME                          READY   STATUS    RESTARTS   AGE
load-generator                1/1     Running   0          43s
nginx-health-demo             1/1     Running   0          10m
nginx-hpa-5b6dcf667b-tzx4g    1/1     Running   0          6m6s
pvc-nginx                     1/1     Running   0          15m
PS D:\internship-celebal\week1\celebal-intern>
```

I configured **Horizontal Pod Autoscaling (HPA)** in my cluster using CPU metrics.
I added resource limits to the deployment, enabled metrics-server, and set autoscaling rules.
Then I stressed the pod to simulate high load and showed how it **scaled up automatically**.
Finally, when the load dropped, the pod count scaled back down, demonstrating autoscaling in action.